STAT 240

Week 4: Markdown and Relational Databases

Owen G. Ward

Today

- Presenting a data science analysis using markdown and Quarto
- Quick introduction to relational databases
- Some basic SQL commands

Announcements

- Midterm Feb 25th,
  - More details coming soon, but if comfortable with labs, in class questions, following code, should be fine
  - Will set up a (timed) practice exam on crowdmark in advance, to show you how it will work, fix possible issues in advance
- A lot of the things we cover today will be useful for avoiding issues in the midterm!

Communicating using Markdown and Quarto

Quarto

- Quarto a relatively new way to write data science analyses up
- All the options of RMarkdown in a more consistent format, allows you to include Python code also
- If familiar with Rmd files will be little difference for this class

The Markdown workflow

- A Qmd file renders a document, and runs R code (the results of the R code can be included in the document)
- A Qmd file is self contained. When you render it runs all code in the document from top to bottom once (and only that code!)
- When preparing a document (or slides), it is good practice to develop in the console and then copy completed code into the markdown file, ensure it runs the way you want

Example

- We saw `set.seed` last week as a way to recreate identical random numbers
- The key step there is that you can replicate things if you run the code in the same order, exactly once
- This is exactly what happens with a Qmd file

Header Code (YAML)

- The first part of a Qmd file is the header, called YAML
- Allows for many options (including headers, footers, scss files)
  - Many of these more useful for making slides
- We will only care about the basics
- Can create a table of contents, etc

Basic header code

```
---
title: "Some title"
author: Some Author
date: 2022-09-12
format: html
---
```

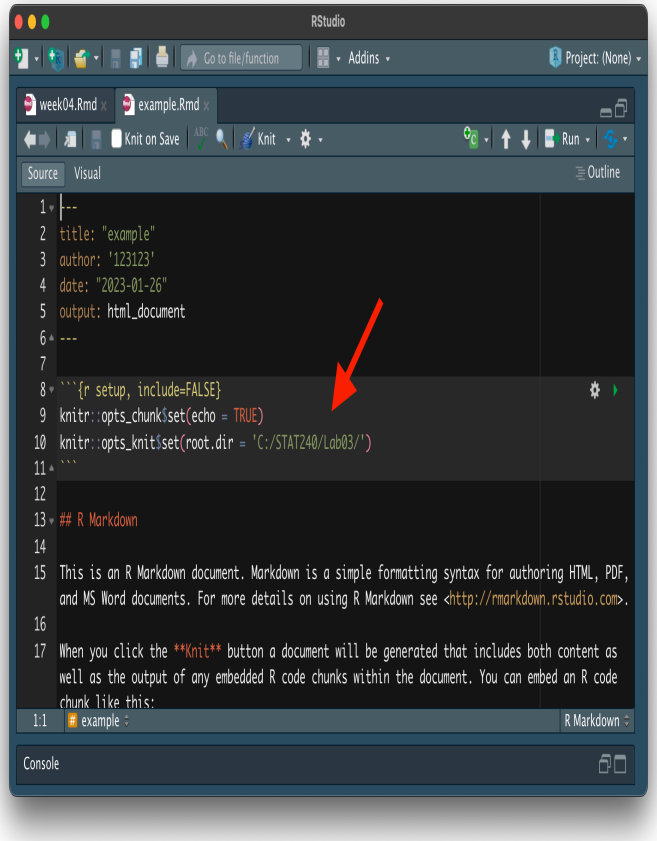
- Requires this format to read the date
- Changing format to pdf requires LaTex (why not required here)
- Will run without this, but no structure

Review: Working directory

- The working directory for the Qmd file is not the same as the working directory for the console (it is a separate *process* on the computer)
- For the Qmd file, the default working directory is the directory that the Qmd is saved in (whereas for the console, the default working directory is your "home" or project directory)

Review: Markdown defaults

- There are several ways to control the working directory in Rmd/Qmd suggested here
- Set *root.dir* in the opening code chunk



Review: Markdown defaults

- If you use an RStudio project (you should) you only need to give the relative path inside the project which is better

````
---
title: "Untitled"
format: html
author: "unknown"
date: 2025-01-22
---

```{r setup}
#| include: false
knitr::opts_knit$set(root.dir = "slides/week_01/")
```


```{r}
data <- read.csv("shot_logs.csv")
head(data)
```
````

Review of Qmd

- The Quarto markdown file is a flat file: You can view and edit in any text editor
- When the document is rendered, special commands in the file indicate formatting and R code
- Extremely flexible format, we will only discuss some of the basics here

Visual and Source Editor

- RStudio includes two versions of the text editor for Qmd files
- The "Visual" editor is maybe easier to start with, similar to things like google docs

- Maybe makes it easier to insert images (not using code chunks), change text style
- Still saves it as a flat file in the background

The Source Editor

- A more plain text way to edit Qmd files
- More similar if you are used to writing R Scripts, Jupyter notebooks
- Can be easier to spot errors in plain text
- Use **Markdown** to format the document, a widely used format

Markdown Syntax

- Examples:

```
# Section title
```

Section title

```
Text, including **bold text** and _italicised text_
```

Text, including **bold text** and *italicised text*

```
- Bullet points
```

- Bullet points

More Examples

- Can use basic TeX to write math

```
Equations:
$$
\sum_{i=1}^n i = \frac{n(n+1)}{2}
$$
```

Equations:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Review of Qmd: More

And `monospace text`

And `monospace text`

- Here ` is the backtick character (on a Canadian keyboard, usually the character to the left of the *1* key—not to be confused with the single quote character ')

Review of Qmd: Code

- Code is often displayed as blocks of monospace text. In Qmd, you can run the blocks, and have the output interleaved into the document
- This is done by having `{r}` next to the opening triple backtick (indicating that the rendering should run the code in R):

````
```{r}
print("Hello, world!")
```
````

```r
print("Hello, world!")
```

```
[1] "Hello, world!"
```

- Each code block in the Qmd can "see" the variables defined in previous blocks (as if the blocks are run one after the other)

````
First block:
```{r}
a <- "Hello, World!"
```


Second block:
```{r}
print(a)
```
````

- You can insert a code chunk using the shortcut `Cmd(Ctrl)/Shift/i`

First block:

```
a <- "Hello, World!"
```

Second block:

```
print(a)
```

```
[1] "Hello, World!"
```

- This makes your code more reproducible!
- To use a variable you must have defined it somewhere above in the qmd file
- Very helpful if you try to run your code a week later, etc

Review of Qmd: Code

- To suppress the code block and only show the output, use the flag `#| echo: false` inside the code chunk
- Lines starting with `#|` read as commands to quarto in how to format things, not R code
- Can change these from chunk to chunk (or set for whole document)

````
```{r}
#| echo: false
print(a)
```
````

```
[1] "Hello, World!"
```

- To prevent the code from being evaluated, use `#| eval: false`

````
First block:
```{r}
#| eval: false
a <- "Introduction to Data Science"
```
Second block:
```{r}
print(a)
```
````

First block:

```r
a <- "Introduction to Data Science"
```

Second block:

```r
print(a)
```

```
[1] "Hello, World!"
```

Using `#| eval: false`

- This is very useful if your qmd file won't render!
- Will stop running if any code gives an error, will stop on the first error!
- Can use this to isolate the errors, see where the problem is exactly
- Keep this in mind for exams!

All the options

Option	Run code	Show code	Output	Plots	Messages	Warnings
`eval: false`	X		X	X	X	X
`include: false`		X	X	X	X	X
`echo: false`		X				
`results: hide`			X			
`fig-show: hide`				X		
`message: false`					X	
`warning: false`						X

Figure 1: From R for Data Science

Exercise

- Create a simple qmd file using `File/New File/Quarto Document`
  - If you created one already during class you can just use that
- Create a chunk with the option `#| error: true` and write some code in that chunk that has an error
- Render the document and see what happens

Global Options

- Often you want to use some of these options for every piece of code
- Older way from RMarkdown is to specify these in the first code chunk

````
```{r setup}
knitr::opts_chunk$set(warning = FALSE, echo = FALSE)
```
````

Global Options in Quarto

- Quarto allows you to do this directly in the YAML at the top also
- Here we are using `knitr` in the background to run the R code, this converts these R specific options to something more general for us

```
---
title: "whatever"
knitr:
  opts_chunk:
    warning: true
    echo: false
---
```

Running code inline

- You can also place small calculations directly in text

```
set.seed(123)
samples <- rnorm(100)
```

- The mean of our `` `r length(samples)` `` samples is `` `r round(mean(samples), 3)` ``.
- The mean of our 100 samples is 0.09.

Code chunk names

- These code chunks have the option of adding (informative) names, which can be used to navigate larger files

````
```{r create a}
a <- "Introduction to Data Science"
```
````

- Will give them default names `chunk 1, chunk 2` if you don't specify them

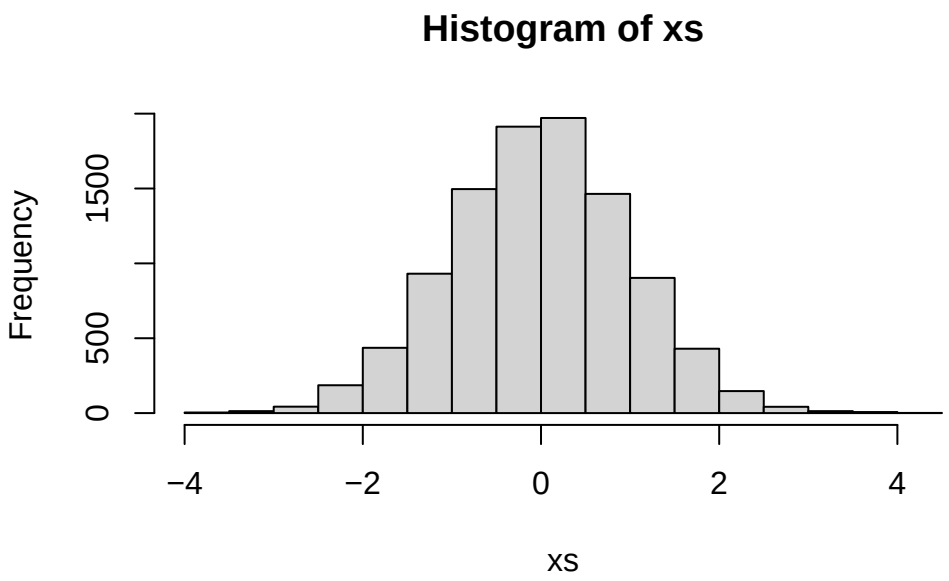
Possible Errors

- These names have to be unique!
- If not your file will not render

Review of Qmd: Plots & Images

- If a code block would produce a plot or image, that plot or image is displayed after the code:

````
```{r}
set.seed(240)
xs <- rnorm(10000)
hist(xs, breaks = 20)
```
````

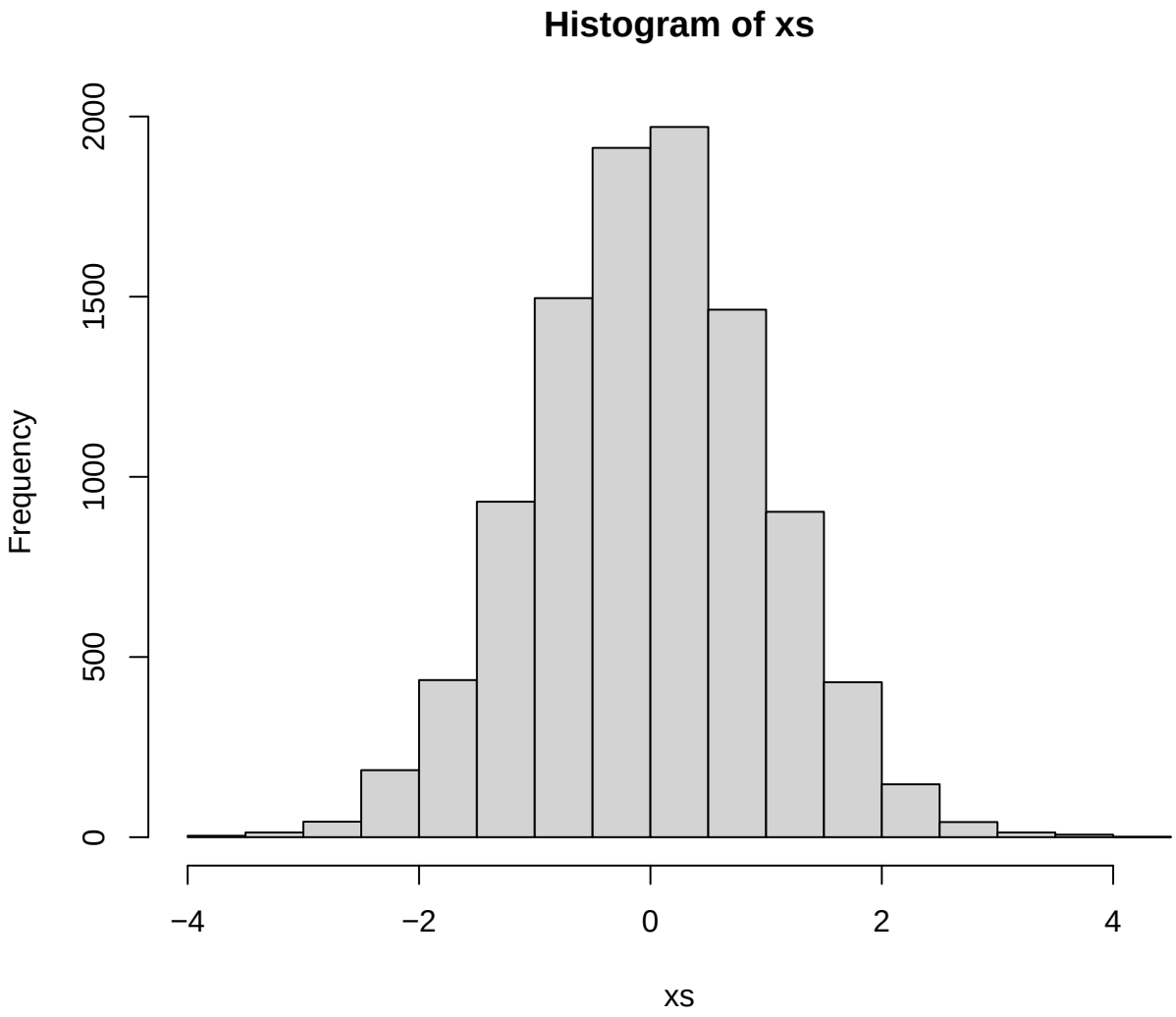


Review of Qmd: Plots & Images

- Default settings for plots may have lower resolution. This can be modified in the code block:

````
```{r}
#| fig-width: 7
#| fig-height: 4
#| fig-dpi: 600
hist(xs, breaks = 20)
```
````

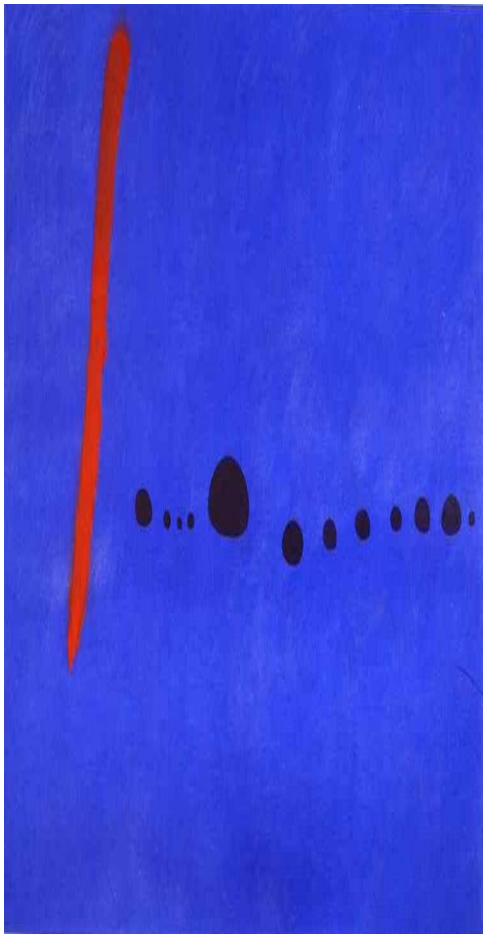
- Useful if you want to use in report, presentation



Review of Qmd: Plots & Images

Code for inserting images:

````
```{r}
library(knitr)
include_graphics("Miro.png")
```
````



Adding Images

- Quarto has lots of additional options for adding in plots outside of doing them in R
- Only downside is can't edit the image

```
![Image Caption](Miro.png)
```

- See the help page for some more examples, options like adding alt text, referencing figures, etc

---

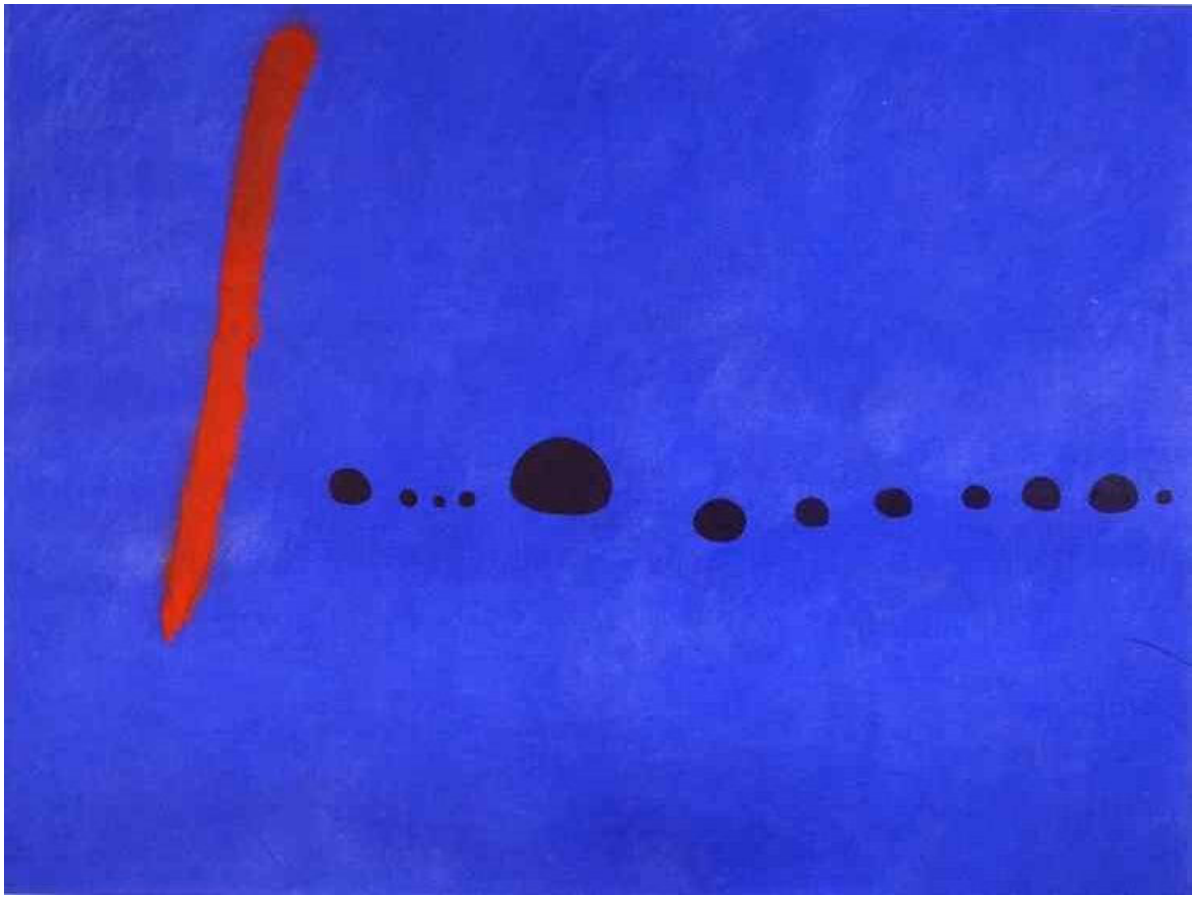


Figure 2: Image Caption

Troubleshooting Qmd Files

- Using a qmd document adds an additional place for errors to occur
- Could have problem with Quarto, or R or both
- Important to learn how to deal with this, it's a common occurance!
- Need to submit a pdf for labs, exams to get graded

Problems with Quarto

- Already mentioned duplicated chunk labels
- Shouldn't need to add too many more options for this class

Problems with R

- If you Qmd doesn't render, good first step is `Restart R and Run all Chunks` under the Run options (top right)
- This will run your code interactively in the console, will stop at the first error
- Common problem is the difference between working directories
- Set `#| eval: false` for the problem chunk to create your report first, then come back to the error (i.e, in exam)

Relational Databases

What are relational databases?

- So far we've looked at small datasets where we can easily manipulate them as flat files
- If you ever work in a company that is actually a data company, this won't be possible!
- We saw information for 27 NASDAQ stocks for one day, minute by minute, which gave 10000 rows

---

- We saw information for 27 NASDAQ stocks for one day, minute by minute, which gave 10000 rows
  - What about all ~7000 companies
  - For every day for the last 50 years
  - At the microsecond level (if you want to actually make money you need better resolution than a second)

```
10000 * 7000/27 * 300 * 50 * 1000000
```

```
[1] 3.888889e+16
```

- This will be TBs of data, RStudio would require this much memory to load it in
- Need something better

Relational databases

- Efficient way of storing and manipulating data. Similar to a data frame in R, but can be synchronized across many locations. Can be reactive, can allow multiple users with different access profiles
- Examples: MySQL, Oracle, Microsoft Access, Postgress
- SQL (Structured Query Language; 1974, IBM)

Relational databases: Advantages

- Availability: Operation can be synchronized over networks between multiple machines or devices
- Integrity: Concurrent modifications can be resolved. Sequences of transactions can be rolled back. Many features for creating and restoring backup
- Security: Multiple users with different access profiles. Encryption & passwords
- Efficiency: Petabyte scale databases, distribution over multiple data centres, low latency

Example use case

- Imagine a department store has begun a franchise
- They now have three locations in the same metro area
- Their inventory is listed on their website, with availability at their locations, and opportunity to buy online
- The website pulls from a SQL database to provide accurate inventory up to the moment (when items are sold at the store, their point of sale software updates the SQL database almost instantly)

The data scientist

- You are hired to do data science, to predict weekly demand using linear regression and yearly trends (next week)
- You should match inventory orders, which arrive weekly.
- On your first day, you're given a username and password to the SQL database for read access, but you can never get to the raw data

Accessing a SQL Database

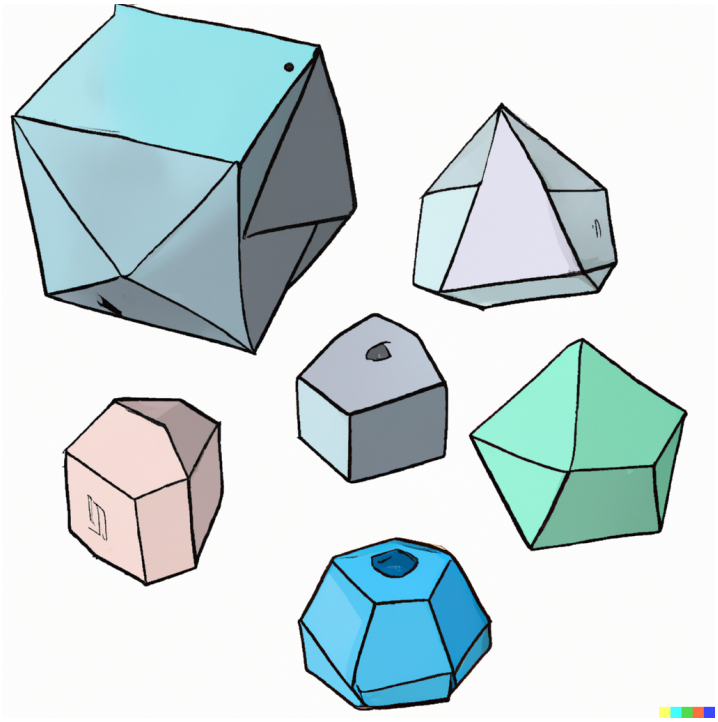
- Several possible ways to get some data from SQL into R
- `dbplyr` a tidyverse style package, allows you to use `dplyr` commands
  - Doesn't require knowing some actual SQL commands, a useful skill
- We will instead use SQLite through the R package `RSQLite` and the `DBI` package (which is loaded by `RSQLite`)

SQL in the wild

- In the real world, might use slightly different version of SQL, slight nuances in the differences
- Focus here on something that is easy to use locally, get a feel for how it works
- Like any tool, have to figure out changes along the way

SQLite

- SQLite requres less configuration than SQL
- File-backed
- R library: *RSQLite*
- A connection is created with the database:



- There is a simple database in the folder for this week, `shapes.sqlite`
- We can open a connection in R to this db using `dbConnect`

```
library(RSQLite)
db <- dbConnect(SQLite(), dbname = "shapes.sqlite")
data <- dbReadTable(db, "Platonic")
print(data)
```

```
         Name Sides
1 Tetrahedron     4
2        Cube     6
```

- We'll spend the next few slides breaking this down some more

Opening the connection

```
db <- dbConnect(SQLite(), dbname = "shapes.sqlite")
```

- We first create a connection to the existing database, and call this `db`, can't really do much with this on it's own

```
db
```

```
<SQLiteConnection>
  Path: /Users/owen/Documents/Github/stat_240/slides/week_04/shapes.sqlite
  Extensions: TRUE
```

- If we didn't specify a file or there isn't one to match the name we specify it will create an empty database

---

- If we want to see what tables of data are in this database we can use `dbListTables(db)`

```
dbListTables(db)
```

```
[1] "Diamonds" "Platonic"
```

Accessing Tables

We can access the data in the DB as a `data.frame` in R using `dbReadTable`, specifying the table name

```
dbReadTable(db, "Platonic")
```

```
         Name Sides
1 Tetrahedron     4
2        Cube     6
```

Adding Datasets

```
dbWriteTable(
  conn = db,
  name = "Diamonds",
  value = ggplot2::diamonds,
  row.names = FALSE,
  overwrite = TRUE
)

head(dbReadTable(db, "Diamonds"))
```

```
  carat       cut color clarity depth table price    x    y    z
1  0.23     Ideal     E     SI2  61.5    55   326 3.95 3.98 2.43
2  0.21   Premium     E     SI1  59.8    61   326 3.89 3.84 2.31
3  0.23      Good     E     VS1  56.9    65   327 4.05 4.07 2.31
4  0.29   Premium     I     VS2  62.4    58   334 4.20 4.23 2.63
5  0.31      Good     J     SI2  63.3    58   335 4.34 4.35 2.75
6  0.24 Very Good     J    VVS2  62.8    57   336 3.94 3.96 2.48
```

Insert a row

- The database is manipulated by passing data through the connection:
- This is done using `SQL` code, so the argument to `dbExecute` is the SQL command require

```
request <- dbExecute(
  db,
  "INSERT INTO Platonic VALUES ('Octahedron', 8)"
)
print(request)
```

```
[1] 1
```

- This returns the number of rows affected by the statement given

An initial Query

- The power of SQL is in accessing specific rows/columns of the data and then just returning those as a data frame
- Smaller and the data you really need

```r
request <- dbSendQuery(
  db,
  "SELECT * FROM Platonic WHERE Sides > 5"
)
data <- dbFetch(request)
dbClearResult(request)
print(data)
```

```
        Name Sides
1       Cube     6
2 Octahedron     8
```

This first Query

- We first write our request for the data we want

```r
request <- dbSendQuery(
  db,
  "SELECT * FROM Platonic WHERE Sides > 5"
)
print(request)
```

```
<SQLiteResult>
  SQL  SELECT * FROM Platonic WHERE Sides > 5
  ROWS Fetched: 0 [incomplete]
       Changed: 0
```

- Then we actually run the query with `dbFetch`

```r
data <- dbFetch(request)
print(request)
```

```
<SQLiteResult>
  SQL  SELECT * FROM Platonic WHERE Sides > 5
  ROWS Fetched: 2 [complete]
       Changed: 0
```

```
dbFetch(request)
```

- This has an optional argument `n` specifying how many rows we want to receive from the database
- Could run this multiple times, if there are more rows we could access (not accessed already)

```
request_new <- dbSendQuery(
  db,
  "SELECT * FROM Platonic WHERE Sides > 5"
)
dbFetch(request_new, n = 1)
```

```
  Name Sides
1 Cube     6
```

```
dbFetch(request_new, n = 1)
```

```
        Name Sides
1 Octahedron     8
```

Exercise

- What is returned if you run `dbFetch(request_new, n = 1)` a third time?

- Then we clear the request for resources, which is a mandatory step

```
dbClearResult(request)
```

- Haven't talked about the actual SQL query code yet!
- Have seen `SELECT` and `INSERT`

SELECT and INSERT

- `SELECT * FROM Name` returns all columns and rows from the table called Name
- `SELECT * FROM Name WHERE ...` returns all columns from the table called Name for rows in which the conditions listed in … are true
- `SELECT V1, V2 FROM Name` returns columns V1 and V2 of all rows in the table called Name
- `INSERT INTO Name VALUES (v1, v2, ...)` Adds the row given by the vector *v1*, *v2*, … into the table called Name

Insert a row without hard coding

```
request <- dbExecute(
  db,
  "INSERT INTO Platonic VALUES ('Octahedron', 8)"
)
```

What if we don't know the exact values we want to insert at time of writing the code? We should create the string programmatically:

```
name <- "Octahedron"
sides <- 8
query <- paste0(
  "INSERT INTO Platonic VALUES ('", name,
  "', ", sides, ")"
)
```

paste0

```
name <- "Octahedron"
sides <- 8
query <- paste0(
  "INSERT INTO Platonic VALUES ('", name,
  "', ", sides, ")"
)
```

- `paste0` is a very handy R function for combining text together

- The `0` means it combines whatever you give it exactly together, doesn't add any spaces, etc

More SQL Queries

- If you've taken Stat 260, the `SELECT` command for SQL might seem familiar
- We can actually do lots of other queries very similar to `dplyr` functions
- I'll just show you some of the key ones here

Ordering Data

- Use `ORDER BY` to get the raw data ordered already
- This could be useful for investigating the max/min

```
request_order <- dbSendQuery(
  db,
  "SELECT * FROM Diamonds ORDER BY price"
)
head(dbFetch(request_order))
```

```
  carat       cut color clarity depth table price    x    y    z
1  0.23     Ideal     E     SI2  61.5    55   326 3.95 3.98 2.43
2  0.21   Premium     E     SI1  59.8    61   326 3.89 3.84 2.31
3  0.23      Good     E     VS1  56.9    65   327 4.05 4.07 2.31
4  0.29   Premium     I     VS2  62.4    58   334 4.20 4.23 2.63
5  0.31      Good     J     SI2  63.3    58   335 4.34 4.35 2.75
6  0.24 Very Good     J    VVS2  62.8    57   336 3.94 3.96 2.48
```

```
dbClearResult(request_order)
```

Largest First

- Just add a `DESC` at the end of the query

```
request_order <- dbSendQuery(
  db,
  "SELECT * FROM Diamonds ORDER BY price DESC"
)
head(dbFetch(request_order))
```

```
  carat       cut color clarity depth table price    x    y    z
1  2.29   Premium     I     VS2  60.8    60 18823 8.50 8.47 5.16
2  2.00 Very Good     G     SI1  63.5    56 18818 7.90 7.97 5.04
3  1.51     Ideal     G      IF  61.7    55 18806 7.37 7.41 4.56
4  2.07     Ideal     G     SI2  62.5    55 18804 8.20 8.13 5.11
5  2.00 Very Good     H     SI1  62.8    57 18803 7.95 8.00 5.01
6  2.29   Premium     I     SI1  61.8    59 18797 8.52 8.45 5.24
```

```
dbClearResult(request_order)
```

Summarising across groups

- A key data science skill is constructing summaries of a table of data across a specific categorical variable
- If you took 260, you would have seen this with `group_by` and `summarise`
- There are very similar options in SQL

Computing Means

```
request_avg <- dbSendQuery(
  db,
  "SELECT cut, COUNT(*) AS n, AVG(price) AS avg_price
  FROM diamonds
  GROUP BY cut"
)
head(dbFetch(request_avg))
```

```
        cut     n avg_price
1      Fair  1610  4358.758
2      Good  4906  3928.864
3     Ideal 21551  3457.542
```

```
4   Premium 13791  4584.258
5 Very Good 12082  3981.760
```

```
dbClearResult(request_avg)
```

Notes on SQL

- SQL will drop NA when you do `GROUP BY` but won't tell you
- Uses `=` not `==` for comparison, because can't assign anything in SQL, only access the data (or insert a row)
- Don't need capitals for SQL commands, but standard to use (makes it easier to read)